

Programación Dinámica

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Repaso de la Clase Anterior



Programación dinámica: Repaso

Anteriormente, en *Programación Dinámica*...

- Enfoque:
  - Bottom-up (PD) vs Top-Down (D&C)
- Requisitos
  - Subestructura Óptima:
    - La solución óptima global se puede construir a partir de las soluciones óptimas de los subproblemas
  - Subproblemas Superpuestos:
    - Resolver uno de los subproblemas requiere la solución de otros subproblemas más chicos resueltos previamente

Programación dinámica: Repaso

Anteriormente, en *Programación Dinámica*...

- Memoization:
  - Guardamos el óptimo de los subproblemas ya obtenidos
  - Clave para reducir la complejidad temporal
- Problemas analizados:
  - Fibonacci
  - Weighted Interval Scheduling
  - Corte de Varilla
  - Materias Ordenadas
  - Caminos posibles de un Laberinto
  - Números Telefónicos

Problema de la Mochila



Problema de la Mochila: Definición

- Se tiene una mochila con una capacidad máxima W
- Se tienen n objetos que ocupan una capacidad w_i y tienen un valor v_i
- Cada objeto puede incluirse en la mochila (si cabe) o ser excluido

Se desea determinar qué elementos poner en la mochila de forma tal que el valor total de los objetos incluidos sea máximo

Ya vimos que los algoritmos Greedy no lograron derrotar a la mochila.
¿Y si probamos con Programación Dinámica?

Problema de la Mochila

Para resolver un problema por PD necesitamos:

- *Dividir el problema en subproblemas menores*
- *Tener una condición de recurrencia*

¿Es posible?



Problema de la Mochila

Subproblemas:

- *Las variables acá son los elementos que incluyo o no en la mochila.*
- *Hagamos una lista ordenada de los elementos y empecemos a guardarlos!*

El primer subproblema es trivial: Nada me impide guardar el primer elemento y tengo un resultado de $\{v_1\}$

Problema de la Mochila

Subproblemas:

El segundo subproblema tiene dos alternativas:

- Si no agrego el elemento, me quedo con la solución del paso anterior: $\{v_1\}$
- Si lo agrego, es posible que tenga que sacar el elemento anterior para guardar éste:
  - Tengo que elegir lo mejor entre $\{v_1, v_2\}, \{v_2\}$

Problema de la Mochila

Subproblemas:
El tercer subproblema se pone más interesante:

- Si no agrego el elemento, me quedo con la mejor solución del paso anterior: $\{v_1\}, \{v_1, v_2\}, \{v_2\}$

- Si lo agrego, es posible que tenga que sacar algún elemento anterior para guardar éste: $\{v_1, v_3\}, \{v_1, v_2, v_3\}, \{v_2, v_3\}, \{v_3\}$

Etc...

Problema de la Mochila

Subproblemas:

- A partir del segundo subproblema ya no se trata solamente de agregar un elemento, sino que también hay que quitar el espacio ocupado por el/los elemento/s previos, y quizás agregar otros previamente descartados.
- En la práctica, es resolver otro problema distinto. No sólo cambian los elementos a colocar, sino el espacio disponible:

$$OPT\,(n,W)$$

Problema de la Mochila

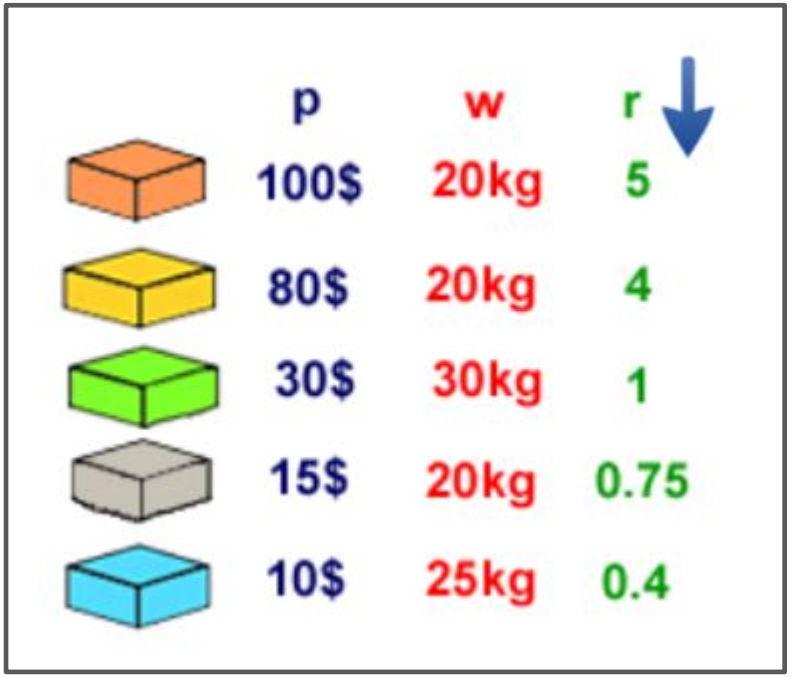
Pasando en limpio:

- $OPT\ (i, W) = max\ (OPT\ (i\text{-}1, W), OPT(i\text{-}1, W\text{-}w_i) + v_i\ )$

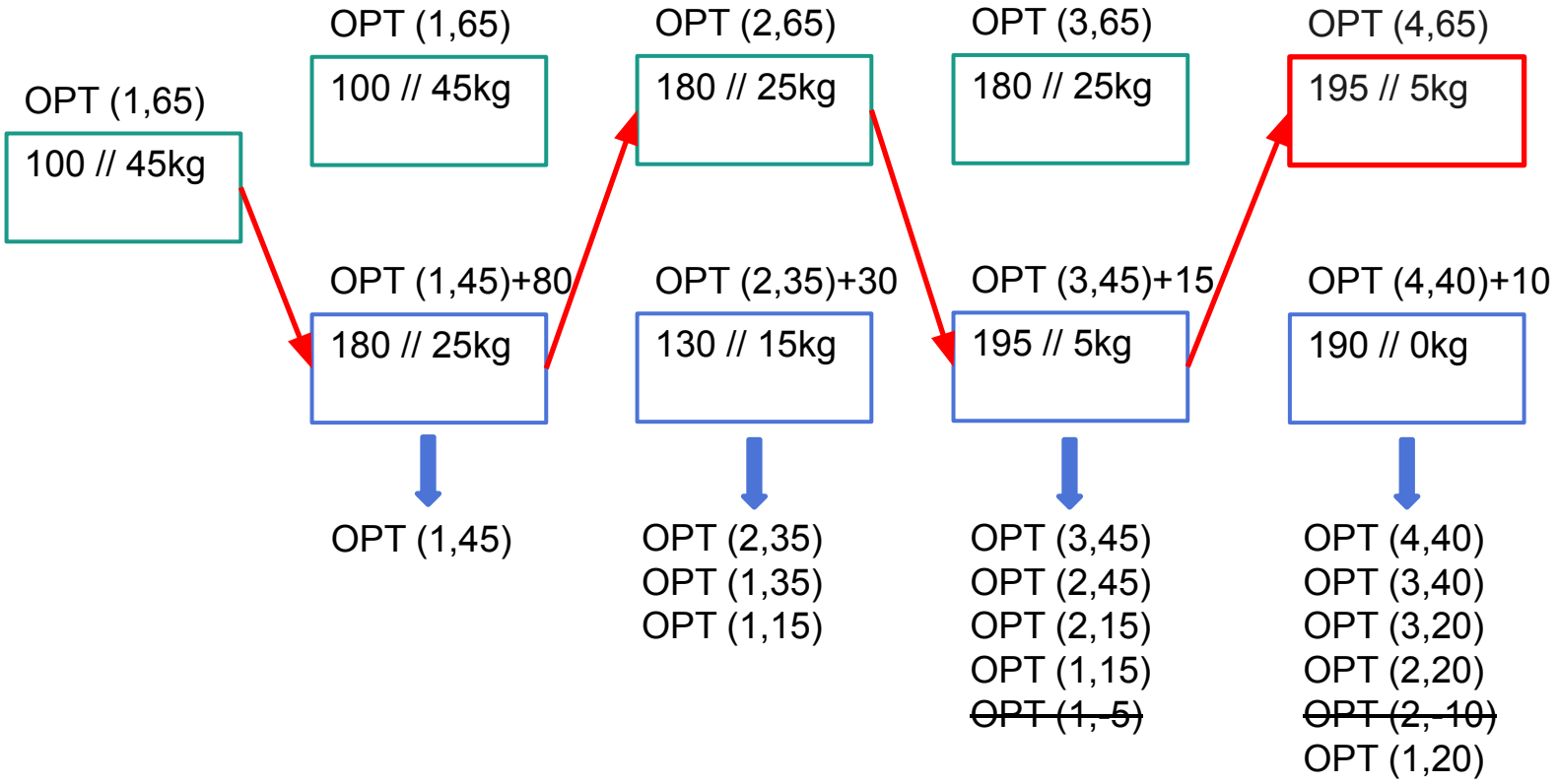
- Ya lo podemos plantear recursivamente.

Problema de la Mochila





Problema de la Mochila



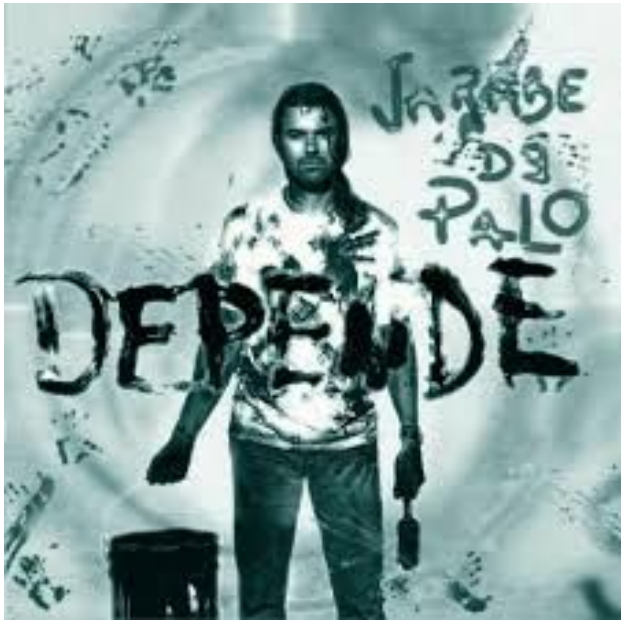
Problema de la Mochila

- ¡Son muchas mochilas!
- ¿Cuánto va a tardar?
- En nuestro ejemplo, hay 5 cantidades de elementos y 13 capacidades diferentes (65 / 5)
- En teoría hay 5 x 13 = 65 mochilas diferentes que calcular.
- Guardamos sus resultados en una matriz para no repetir los cálculos después.
- Para calcular cada mochila, sólo hay que comparar $OPT\ (i\text{-}1, W)$ con $OPT\ (i\text{-}1, W\text{-}w_i)+v_i$. Eso es lineal.
- Este algoritmo es de $O\ (n.W)$



Problema de la Mochila

- Este algoritmo es de $O\,(n.W)$
- ¿Eso significa que es polinomial?

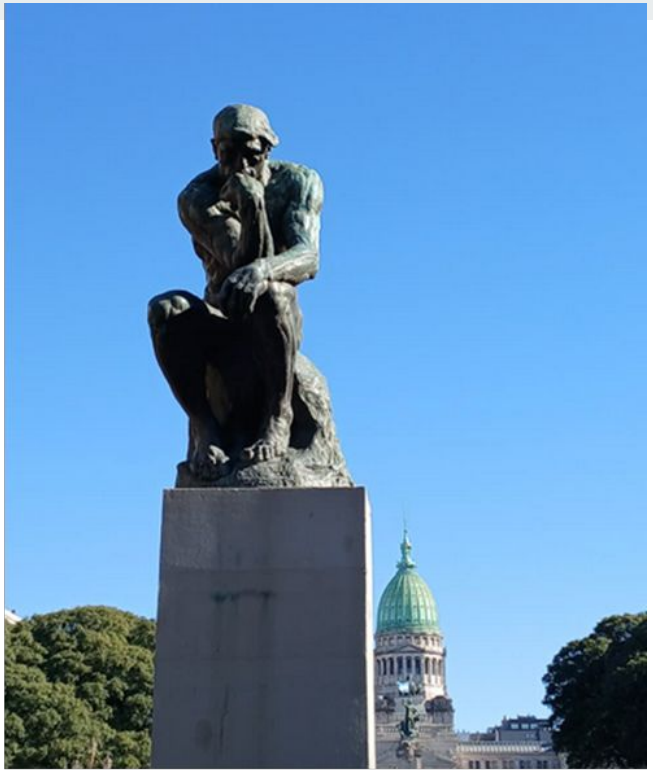


Problema de la Mochila: Complejidad Temporal

- Este algoritmo es de $O\,(n.W)$
- ¿En función de qué medimos la complejidad del problema?
- W es un valor numérico que representa el peso de cada elemento.
- Ingresa al programa no como un valor único, sino como una serie de bits (por ejemplo, 65 = 1000001 ó 13 = 1101)
- ¿Cuántos bits se necesitan para representar a W?
- $b = log_2\ W$, ya que $W=2\text{^}b$

Problema de la Mochila: Complejidad Temporal

- Este algoritmo es de $O\,(n.W)$
- También podemos decir que es $O\,(n.2^b)$
- ¿Es polinomial?



"El Pensador" - A. Rodin

Problema de la Mochila: Conclusiones

- Este tipo de problemas se suelen llamar pseudo-polinomiales.
- Si uno lo expresa en función del valor numérico de la entrada, es polinomial.
- Si lo expresa en función de la longitud de dicha entrada, es exponencial.
- ¿Podemos decir que derrotamos eficientemente al problema de la mochila?
- Continuará…….

Problema de la Mochila: Formulación alternativa

- Si los valores de los pesos son grandes, esto puede generar muchos subproblemas.
- Si los pesos son grandes y los beneficios son pequeños, se puede plantear el problema en función de los beneficios y no de los pesos.
- En lugar de “¿Cual es el beneficio máximo que puedo obtener con una mochila de capacidad W?”, planteamos la pregunta “¿Cuál es la mochila más liviana con la que podemos obtener un beneficio V?”
- Si vamos incrementando el valor de V, la solución será la última mochila cuya capacidad sea menor o igual que W.

Cambio de Monedas



Pixabay

Cambio de Monedas: Definición

- Se tiene un sistema monetario S formado por n monedas $S=\{m_1, m_2, \ldots, m_n\}$

Encontrar la mínima cantidad de monedas necesarias para sumar cierto monto de dinero

Cambio de Monedas: Análisis Greedy

Regla Greedy:

Usar la moneda de mayor denominación posible, todas las veces que se pueda

Cambio de Monedas: Análisis Greedy (cont.)

Sistema Monetario $S=\{5,10,20,25\}$

- Monto: 45: Solución: una moneda de 25, una moneda de 20.
  - Es una solución óptima
- Monto: 40. Solución: una moneda de 25, una de 10, una de 5.
  - O mejor, dos monedas de 20

La optimalidad del algoritmo Greedy depende del sistema monetario

Cambio de Monedas: Análisis PD

¿Cómo podemos plantear algo que cumpla con las condiciones de PD?

- Subestructura Óptima: ¿qué problema se puede basar en la solución del mismo problema pero más pequeño?
- Subproblemas Superpuestos: ¿de qué manera podemos usar subproblemas ya resueltos en una iteración previa?

¿Cuál sería el subproblema que debemos tener resuelto previamente para poder encontrar la solución al problema original?

¿Cuál sería su caso base?

Cambio de Monedas: Análisis PD (cont.)

Analicemos cómo lo hacemos en la vida diaria para un monto dado:

1. Repetir:
    a. Tomar la moneda de mayor denominación posible
    b. Restar el valor de la moneda tomada al monto
2. Mientras quede saldo a pagar

Cambio de Monedas: Análisis PD (cont.)

Pero debemos encontrar la solución que requiera la cantidad mínima de monedas:

1. Repetir para cada moneda del sistema:
    a. Tomar la próxima moneda
    b. Calcular el saldo restando el valor de la moneda tomada al monto
    c. Ver la mejor manera de pagar el saldo
    d. Sumar 1 (porque hay que agregar la moneda actual)
    e. Si la cantidad total de monedas es menor que la mejor que teníamos hasta ahora, guardar la moneda elegida y seguir

Cambio de Monedas: Análisis PD (cont.)

Sistema Monetario={5, 10, 20, 25}

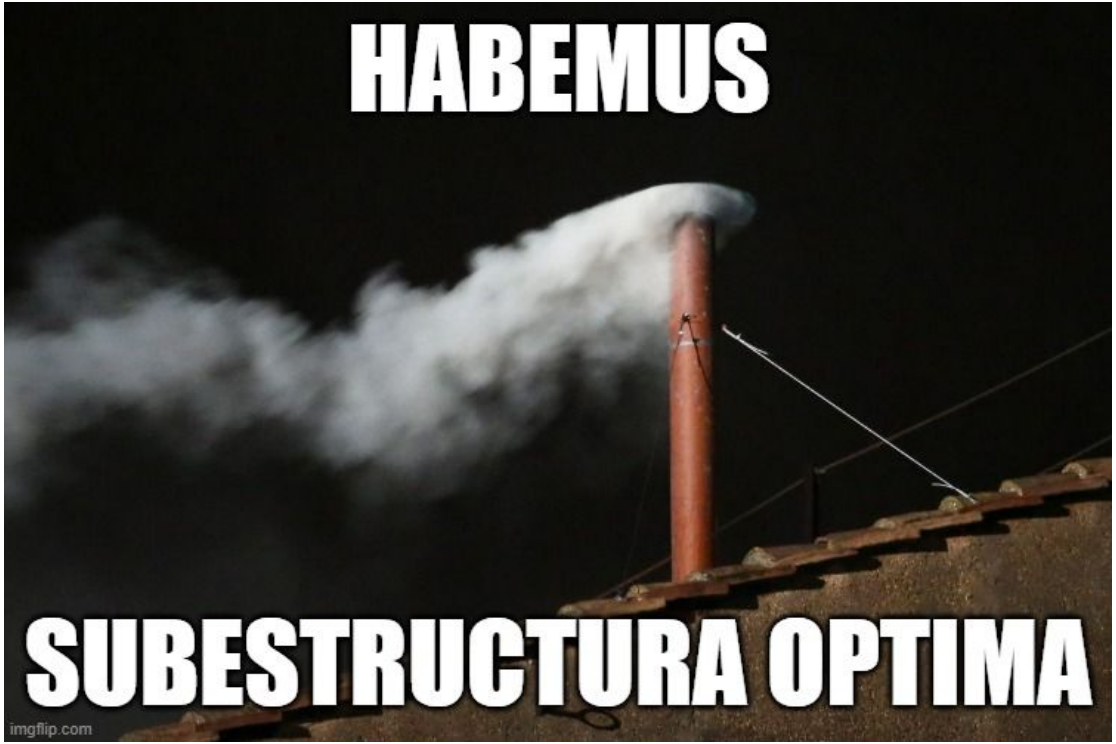
- Monto=0: devolver 0 monedas de 0; resto=0
- Monto=5: devolver 1 moneda de 5; resto=0
- Monto=10: devolver 1 monedas de 10; resto=0
- Monto=20: devolver 1 moneda de 20; resto=0
- Monto=30: devolver 1 moneda de 25; resto=5; devolver una moneda de 5; resto=0 (también podría haber sido una de 20 y otra de 10)
- Monto=40: devolver 1 moneda de 20; resto=20; devolver 1 moneda de 20; resto=0

Cambio de Monedas: Análisis PD (cont.)

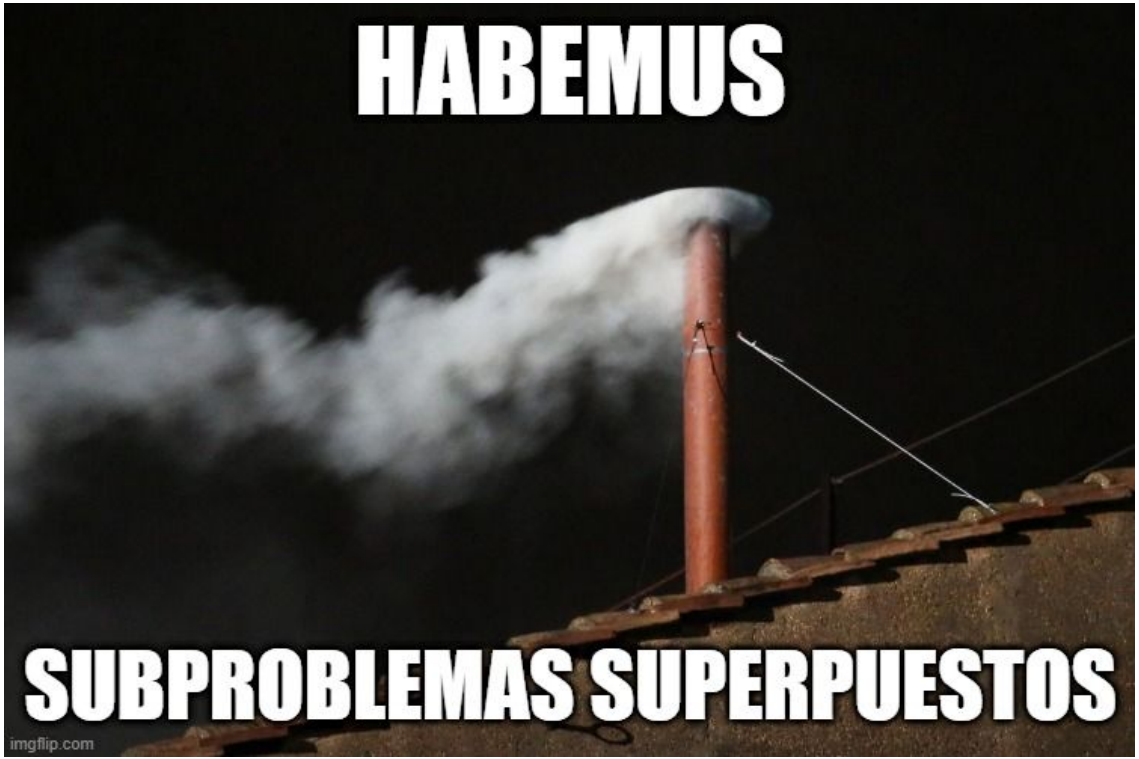
Establezcamos la ecuación de recurrencia para un sistema monetario $S=\{m_1, m_2, ..., m_n\}$:

$$OPT(monto) = 1 + min \begin{cases} OPT(monto - m_1); \\ OPT(monto - m_2); \\ \cdots \\ OPT(monto - m_n) \end{cases}$$

Cambio de Monedas: Análisis PD (cont.)



Cambio de Monedas: Análisis PD (cont.)



Cambio de Monedas: Programación Dinámica

```
def cant_monedas(sist_monetario, monto):
  cant = [0] * (monto + 1)
  for i in range(1, monto + 1):
    minimo = i  # usar todas monedas de 1
    for moneda in sist_monetario:
      if moneda > i:
        continue
      cantidad = 1 + cant[i - moneda]
      if cantidad < minimo:
        minimo = cantidad
    cant[i] = minimo
  return cant[monto]
```

Gentileza Curso Buchwald

Cambio de Monedas: Programación Dinámica

¿Cuál es la complejidad del algoritmo presentado?

- Primer loop: se repite n veces (según el monto)
- Segundo loop: se repite m veces (según la cardinalidad del sistema)

$\therefore$ Complejidad Temporal: $O(n \; x \; m)$

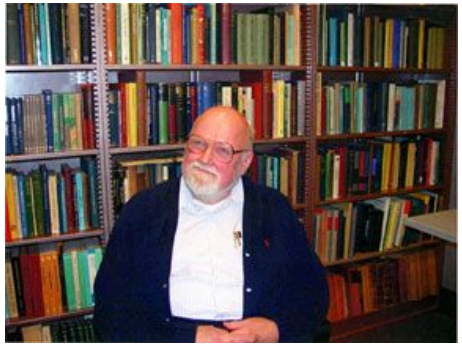
¿Es pseudopolinomial?

¡Sí! Porque podemos ver a n cumpliendo la misma función que W en el problema de la mochila

Algoritmo de Bellman-Ford



Richard Ernest Bellman (1920-1984)
Créditos: https://mathshistory.st-andrews.ac.uk/Biographies/Bellman/poster/lived/

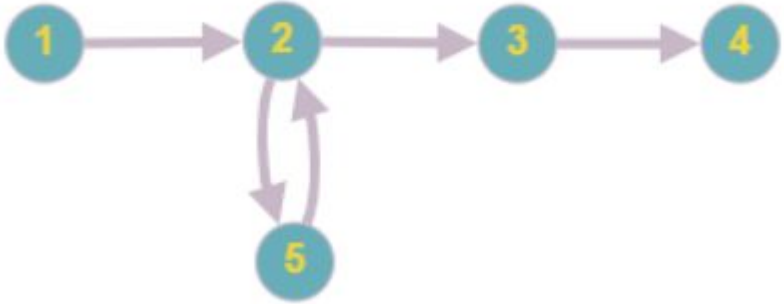


Lester Ford Jr. (1927-2017)
Créditos: https://optimizacioniimac.blogspot.com/2019/04/lester-randolph-ford-jr.html

Bellman-Ford: Descripción (cont.)

- ¿Cómo detecta ciclos negativos?
  - Porque el camino de menor distancia entre dos vértices es el camino simple (no se repiten vértices) y mide a lo sumo $|V|-1$ aristas
  - Si hay un ciclo negativo, significa que conviene volver a visitar vértices que ya visitó, entonces el camino tendrá distancia mayor o igual a $|V|+1$





Bellman-Ford: Análisis

Dijkstra era un algoritmo Greedy

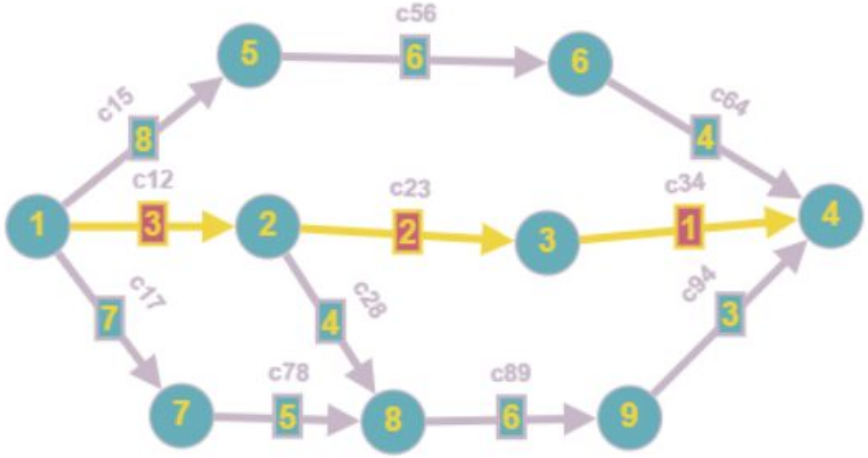
¿Bellman-Ford es un algoritmo de Programación Dinámica?

Bellman-Ford: Análisis (cont.)

- En la primer iteración, busca todos los caminos P de longitud $i=0$ partiendo del vértice origen v:
  - La distancia al vértice origen es 0
  - La distancia a todo el resto de los vértices es infinito (no hay forma de llegar a los otros vértices atravesando 0 aristas)
- En la segunda iteración, busca todos los caminos P de longitud $i=1$, partiendo del vértice origen v:
  - La distancia al vértice origen es 0
  - La distancia a los vecinos de v es 1
  - La distancia a todo el resto de los vértices es infinito

Bellman-Ford: Análisis (cont.)

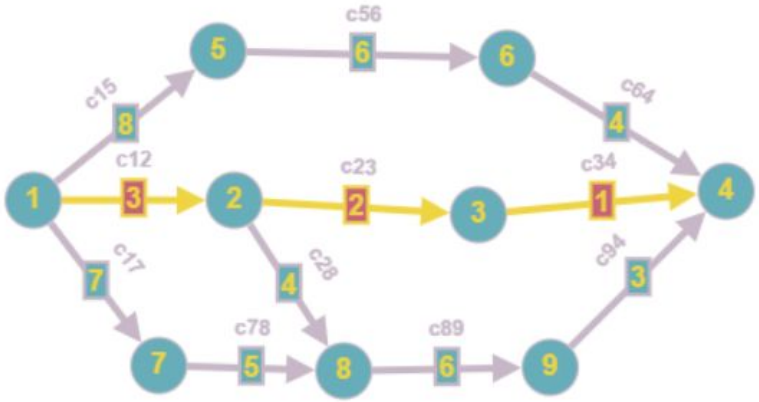
La mejor forma de llegar al vértice destino t desde v es un camino cuyo peso es el mínimo para cada arista vecina de t más el peso del camino mínimo que recorre desde el origen v hasta esa arista en un paso menos



Bellman-Ford: Análisis (cont.)

Definamos $OPT(i, v)$ como el mínimo costo para ir desde un vértice v hacia un vértice t usando *a lo sumo* i aristas:

- Si existe un camino P óptimo entre v y t y usa a lo sumo i-1 aristas, entonces $OPT(i,v)=OPT(i-1, v)$



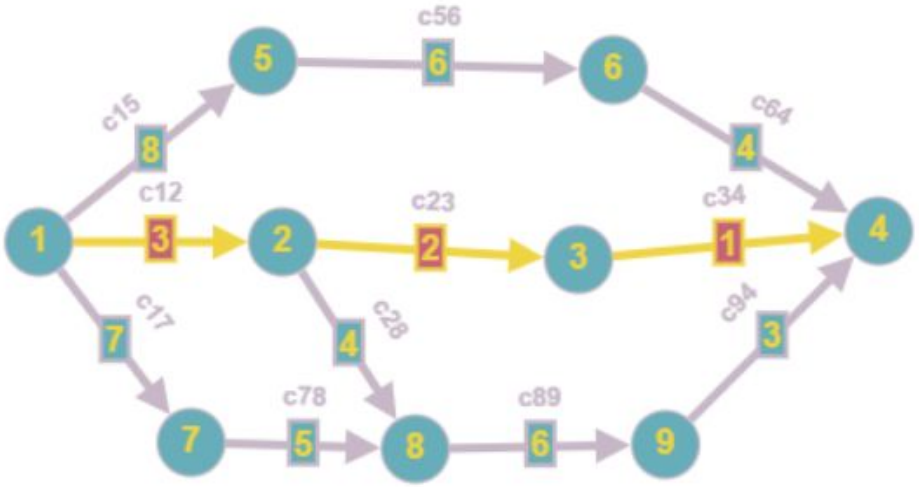
Queremos ir de 1 a 4 ($v=1$; $t=4$):

$P=\{1,2,3,4\}$

- $OPT(2,1)=\infty$
- $OPT(3,1)=6$
- $OPT(4,1)=6$

Bellman-Ford: Análisis (cont.)

- Si P usa i aristas y la primer arista es v-w, entonces $OPT(i,v)=c_{vw}$ + $OPT(i-1, w)$



$P=\{1,2,3,4\}$

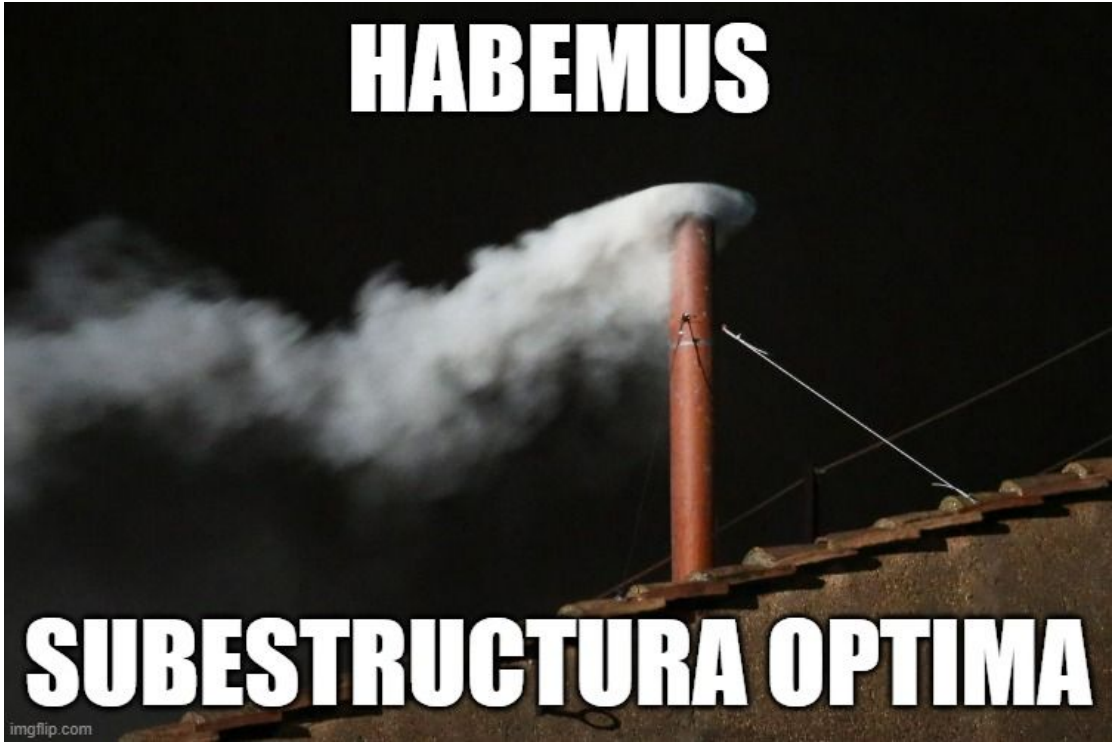
- $OPT(3,1)=c_{12}+OPT(2,2)$

Bellman-Ford: Análisis (cont.)

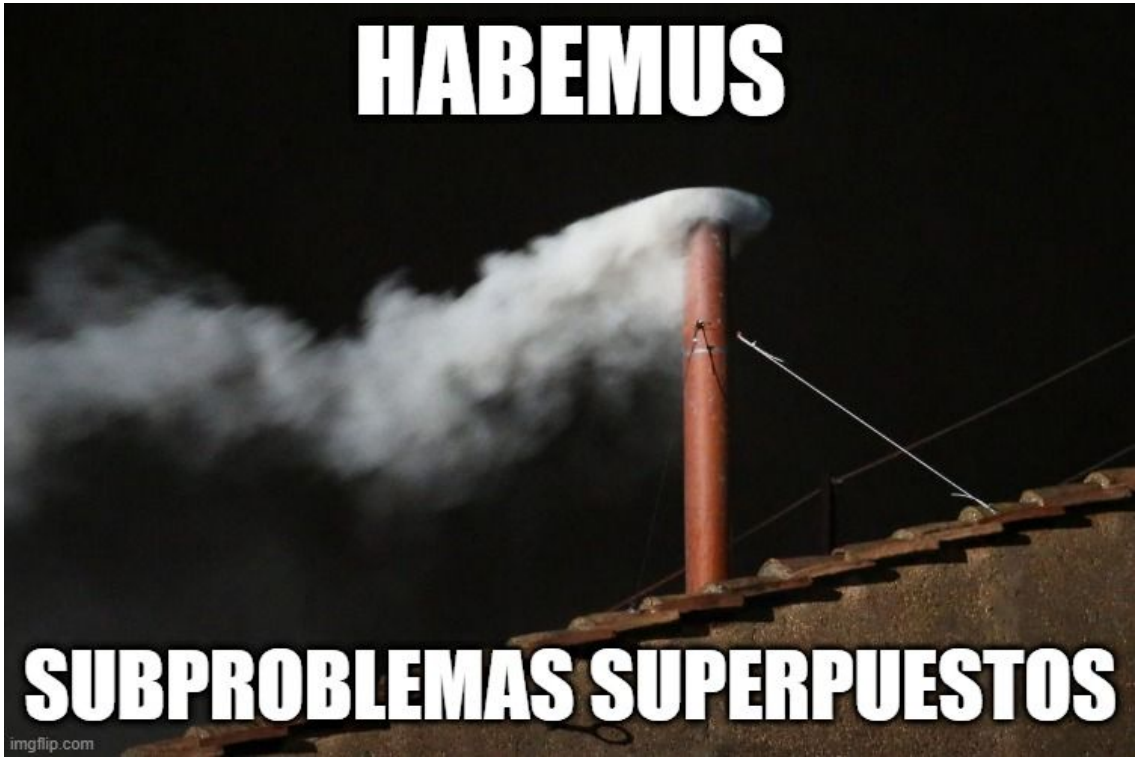
La ecuación de recurrencia es:

$$OPT(i, v) = min(\quad OPT(i-1, v), \quad \min_{w \in V}(c_{vw} + OPT(i-1, w)))$$

Bellman-Ford: Análisis (cont.)



Bellman-Ford: Análisis (cont.)



Bellman-Ford: Pseudocódigo

```
Desde i=0 a n-1
  OPT[i][0] = 0

OPT[0][t]=0 // peso de t a sí mismo es cero

Desde v=1 a n
  OPT[0][v] = +Infinito

Desde i=1 a n-1
  Desde v=1 a n
    OPT[i][v] = OPT[i-1][v]
    Por cada p sucesor de v
      si OPT[i][v] > OPT[i-1][p] + w(v,p)
        OPT[i][v] = OPT[i-1][p] + w(v,p)

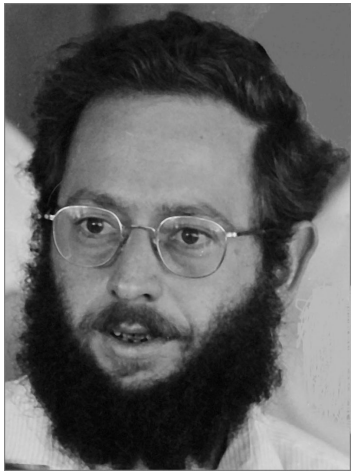
Retornar OPT[n-1,s]
```

Gentileza Curso Podbereszki

Algoritmo de Floyd-Warshall



Robert W. Floyd (1936-2001)
Créditos: Stanford University News Service



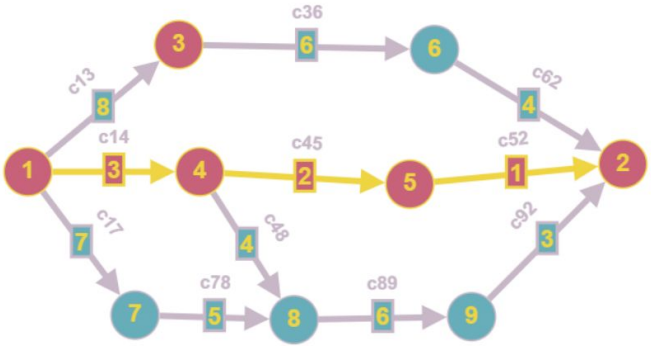
Stephen Warshall (1935-2006)
Créditos: Robert M. McClure

Floyd-Warshall: Descripción

- Se tiene un grafo $G=(V, E)$ dirigido y ponderado, sin ciclos negativos
- Admite pesos negativos
- Determina el camino mínimo $\forall\, i, j \in V$

Floyd-Warshall: Análisis

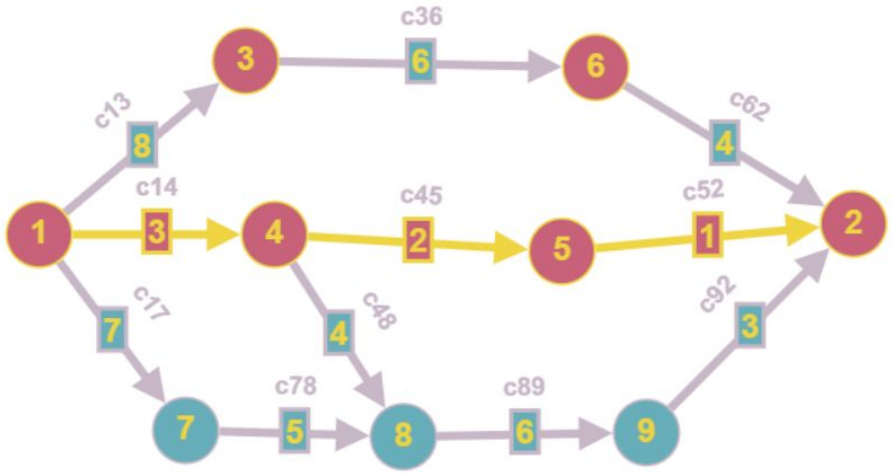
- Enumeramos los vértices del grafo de 1 a n
- Definimos $S_k=\{v_1, v_2, \ldots, v_k\}$ como el subconjunto de V que contiene los primeros k vértices de V según la enumeración anterior
- Sea $P(i,j,k)$ un camino mínimo de v_i a v_j usando vértices en S_k
- Los vértices v_i, v_j no necesariamente deben estar en S_k
- Un vértice v es intermedio si v está en $P(i,j,k)$ y en S_k y no es ni v_i ni v_j



- $k=5$
- $S_5=\{1,2,3,4,5\}$
- $P(1,2,5)=\{1,4,5,2\}$
- $4, 5$ son vértices intermedios de $P(1,2,5)$

Floyd-Warshall: Análisis (cont.)

Si v_k no es vértice intermedio de $P(i,j,k)$, entonces $P(i,j,k)=P(i,j,k-1)$, y también es un camino mínimo



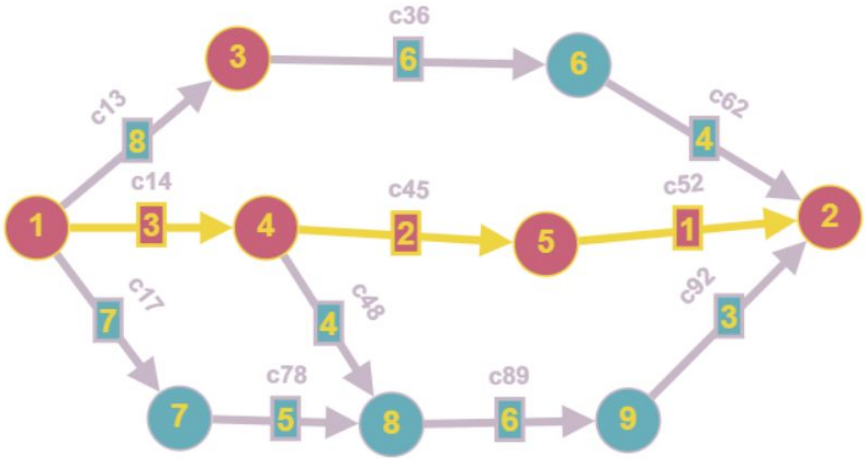
- $k=6$
- $S_6=\{1,2,3,4,5,6\}$
- $P(1,2,6)=\{1,4,5,2\}$

- v_6 no es intermedio de $P(1,2,6) \Rightarrow P(1,2,6)=P(1,2,5)$

Floyd-Warshall: Análisis (cont.)

Si v_k *es un vértice intermedio de* $P(i,j,k)$:

- $\nexists\, P(i,j,k\text{-}1) \,/\, v_k \in P(i,j,k\text{-}1)$ porque v_k no pertenecería a S_k (aunque podría existir otro camino de v_i a v_j usando $S_{k\text{-}1}$)



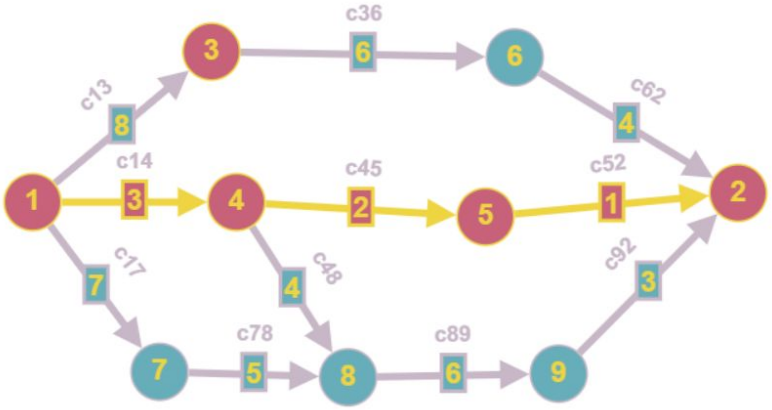
- $k=5$
- $S_5=\{1,2,3,4,5\}$
- $P(1,2,5)=\{1,4,5,2\}$

- v_5 es intermedio de $P(1,2,5)$:
  - $\nexists\, P(1,2,4) / v_5 \in P(1,4,4)$

Floyd-Warshall: Análisis (cont.)

Si v_k es un vértice intermedio de $P(i,j,k)$:

- Podemos descomponer el camino en dos: $P(i,k,k-1)$ y $P(k,j,k-1)$, que además son óptimos en S_{k-1} (de lo contrario, $P(i,j,k)$ no hubiera sido óptimo en S_k)



- $k=5$
- $S_5=\{1,2,3,4,5\}$
- $\nexists P(1,2,4) \Rightarrow v_5$ es intermedio de $P(1,2,5)$
- $P(1,5,4)=\{1,4,5\}$
- $P(5,2,4)=\{5,2\}$

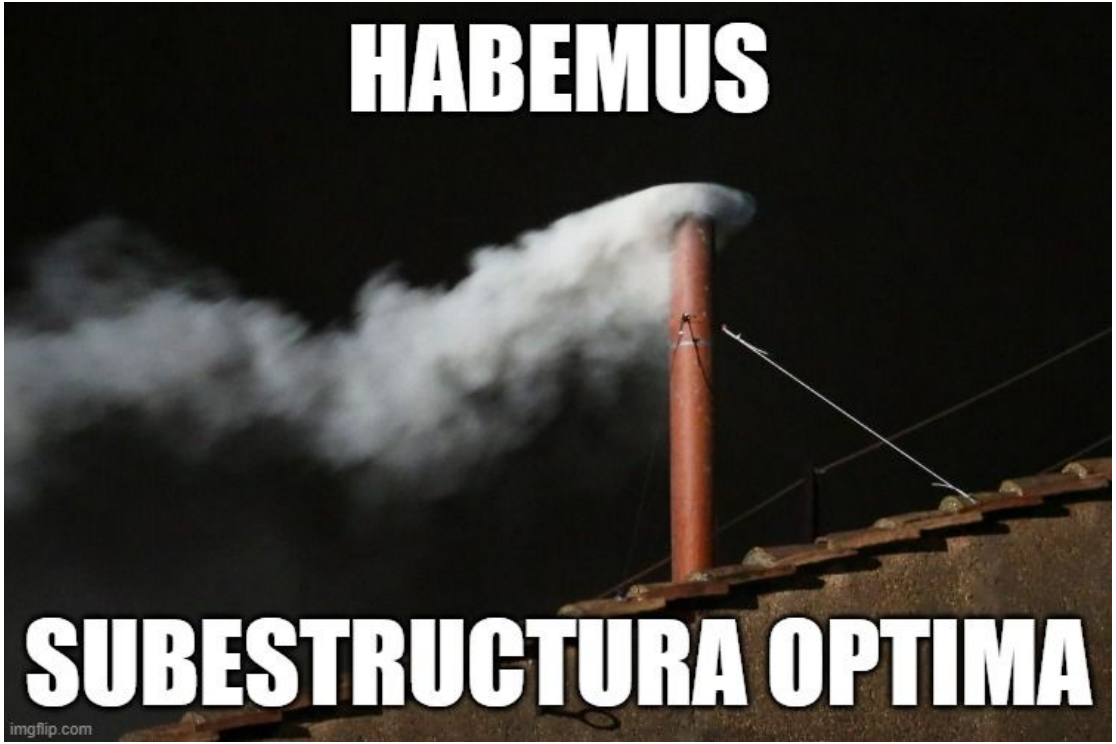
- $P(1,2,5)=P(1,5,4) + P(5,2,4)$

Floyd-Warshall: Análisis (cont.)

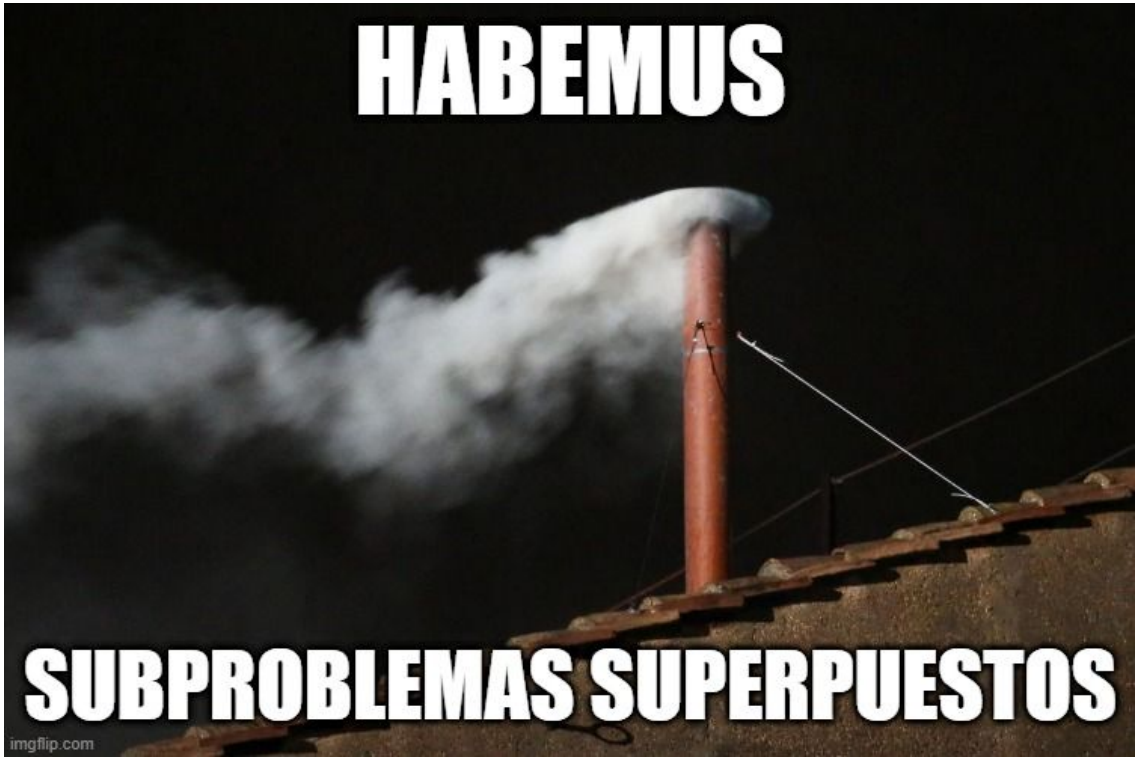
Ecuación de Recurrencia:

$$OPT(i,j,k) = \begin{cases} w_{ij} & \text{si } k = 0 \\ min \begin{pmatrix} OPT(i,j,k-1); \\ OPT(i,k,k-1) + OPT(k,j,k-1) \end{pmatrix} & \text{si } k >= 1 \end{cases}$$

Floyd-Warshall: Análisis (cont.)



Floyd-Warshall: Análisis (cont.)



Floyd-Warshall: Pseudocódigo

```
Flowyd_Warshall(W)
  P[0]=W
  n=|V|
  Para k=1 a n
    sea P[k] una matriz de nxn
    Para i=1 a n
      Para j=1 a n
        P[k][i,j]=min(P[k-1][i,j],
                      P[k-1][i,k] + P[k-1][k,j])
  Devolver P[n]
```

Gentileza Curso Podbereszki

Floyd-Warshall: Reconstrucción del Camino

Caso Base:

$$C^0_{i,j} = \begin{cases} null & \text{si } i = j \vee w_{i,j} = \infty \\ i & \text{si } i \neq j \wedge w_{i,j} < \infty \end{cases}$$

Caso General:

$$C^k_{i,j} = \begin{cases} C^{k-1}_{i,j} & \text{si } P(i,j,k-1) \leq P(i,k,k-1) + P(k,j,k-1) \\ C^{k-1}_{k,j} & \text{si } P(i,j,k-1) > P(i,k,k-1) + P(k,j,k-1) \end{cases}$$

Gentileza Curso Podbereszki



¡Muchas gracias!